

BruFace

MA1 Mechanical Engineering
Vibrations and Acoustics
Academic year 2022-2023

Session 3 : The offshore wind turbine

Welcome to the third session

During the practical session, we will use the same libraries as the one used in the previous sessions. But we'll add a new package called `pandas`. `pandas` is a very useful package when it comes to importing and processing big datasets. It can handle all types of data and today we will use it to import measurement data.

Don't forget: It is important that you run each cell of the notebook. To do so select a cell (it will be highlighted) and press shift+enter on your keyboard or the play button in the menu above.

```
In [1]: # let's import both numpy and matplotlib.pyplot
# the "as" in the code bellow is to tell python that if we reference
import numpy as np
import matplotlib.pyplot as plt
import scipy.linalg as linalg
import pandas as pd
# don't worry about these lines
%matplotlib notebook
from IPython.display import import YouTubeVideo
```

Exercises

Context

In today's session we will analyse the measurement data from an offshore wind turbine. The measurements that were shared are from two accelerometers installed in the nacelle of the wind turbine. The nacelle is the box at the top of the tower that houses the main drive shaft, gearbox and generator. We call the direction aligned with the wind direction the Fore-Aft direction (FA) and the direction cross-wind the sideways (SS) direction.

We will use the measurement to analyze the dynamic properties of the wind turbine and investigate its susceptibility to different types of loading.

Figure 1: Two offshore wind turbines



Exercise 1: the rotor stop

To start the exercise we will first import some data from an actual offshore wind turbine in the North sea. For this purpose we will use the package `pandas` and the `read_csv` command to read data from a csv file with the data stored in it. Once imported the data is stored in a so-called `Dataframe` (hence why we call it `df`). Using the `df.head()` command we show the first five lines of the `Dataframe`.

```
In [2]: df = pd.read_csv('offshore_wind_data_rotor_stop.csv', header=0)
df.head()
```

```
Out[2]:
```

	t [s]	FA [g]	SS [g]
0	0.00	-0.004464	0.009247
1	0.04	-0.003821	0.007874
2	0.08	-0.004222	0.007740
3	0.12	-0.004667	0.008032
4	0.16	-0.004643	0.009867

Assignment 1.1: Use the cell below to find key information about the measurement dataset. Define:

- The sample frequency in Hz: `f\_s`
- The duration in s: `t\_e`
- The measurement unit of the acceleration: `unit\_str`

```
In [3]: ###BEGIN SOLUTION
f_s = 25 #Hz
t_e = 600 #s
unit_str = 'g'
###END SOLUTION
```

Some small checks

```
In [4]: assert f_s*t_e == len(df) # See if the total number of samples adds u
```

Assignment 1.2: use the `plt.plot` command to visualize the measurement data

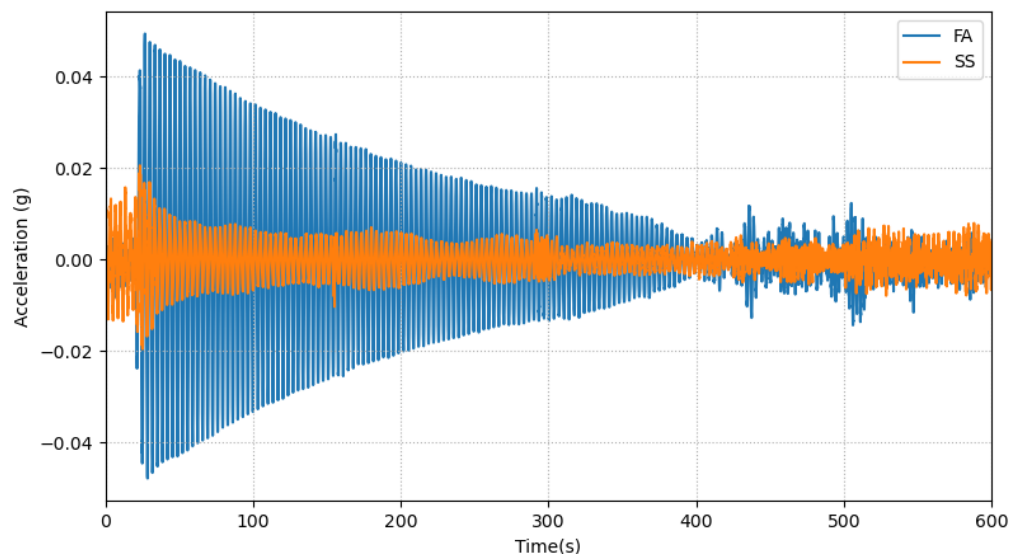
```
In [5]: plt.figure(figsize=(9,5))

###BEGIN SOLUTION
plt.plot(df['t [s]'],df['FA [g]'], label='FA',zorder=1)
plt.plot(df['t [s]'],df['SS [g]'], label='SS',zorder=2)

plt.xlim([0,600])
plt.legend()
plt.grid(which='both', linestyle=':')
plt.xlabel('Time(s)')
plt.ylabel(f'Acceleration ({unit_str})')
###END SOLUTION

plt.show()
```

<IPython.core.display.Javascript object>



In this dataset the turbine performs a so-called **rotor stop**, during a rotor stop the turbine shuts down and the wind load is released. In this video you can see an onshore wind

turbine do a rotor stop:

```
In [11]: YouTubeVideo('_qVXkAwH60', width=800, height=300)
```

Out[11]:

In a way you could look at this rotor stop as an impulse response, so let us try to determine some of the wind turbine properties from it.

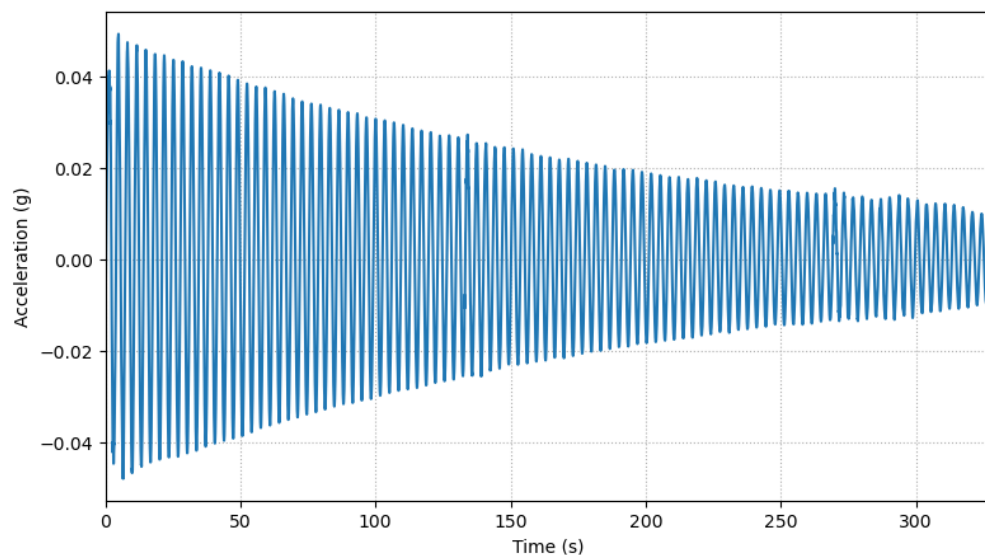
Assignment 1.3: Can you isolate the data from the rotor stop in the FA direction? Use `df.to_numpy` to extract data from a dataframe into a numpy array (which is what we used in the previous sessions). Store your variable as `fa_rotor_stop`, also store the corresponding timevector (t=0s, on the rotor stop) in `t_rotor_stop`

```
In [7]: ###BEGIN SOLUTION
fa_rotor_stop = df['FA [g]'].iloc[22*f_s:350*f_s]
t_rotor_stop = np.array(range(len(fa_rotor_stop)))/f_s
###END SOLUTION

plt.figure(figsize=(9,5))

plt.plot(t_rotor_stop, fa_rotor_stop)
plt.xlim([t_rotor_stop[0], t_rotor_stop[-1]])
plt.show()
plt.xlabel('Time (s)')
plt.ylabel('Acceleration (g)')
plt.grid(which='both', linestyle=':')
```

<IPython.core.display.Javascript object>



Assignment 1.4: Convert the impulse response data to the frequency domain using `np.fft.fft` and `np.fft.fftfreq` and plot the resulting spectra. Use these spectra to estimate the first natural frequency of the structure.

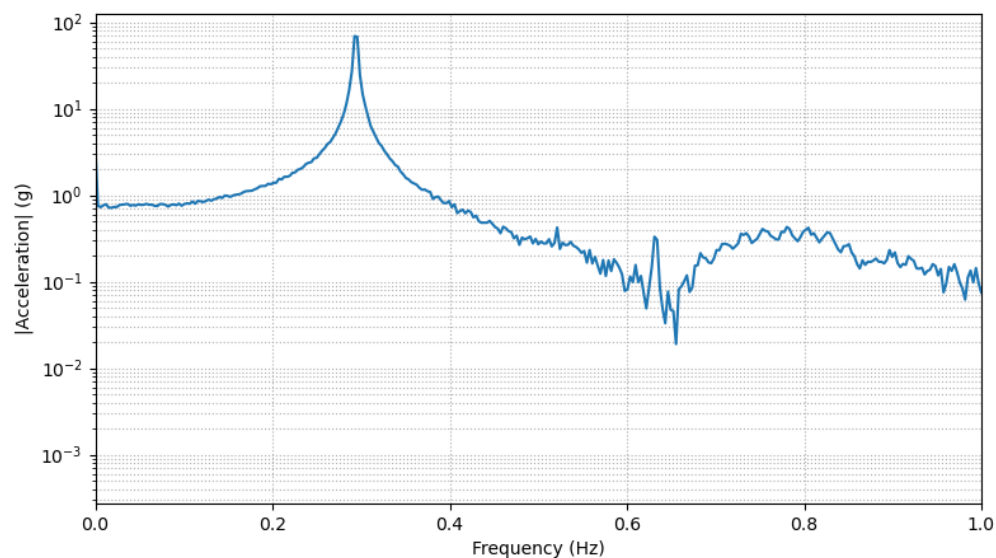
```
In [8]: plt.figure(figsize=(9,5))

####BEGIN SOLUTION
f = np.fft.fftfreq(len(t_rotor_stop), 1/f_s)
X = np.fft.fft(fa_rotor_stop)

fh = int(len(f)/2)
plt.semilogy(f[:fh], np.abs(X[:fh]))
plt.xlim([0,1])
####END SOLUTION

plt.xlabel('Frequency (Hz)')
plt.ylabel('|Acceleration| (g)')
plt.grid(which='both', linestyle=':')
plt.show()
```

<IPython.core.display.Javascript object>



What resonance frequency do you conclude for this structure? Set a variable `w_n` accordingly (best in rad/s)

```
In [9]: ####BEGIN SOLUTION
w_n = 0.295 * 2 * np.pi # rad/s
####END SOLUTION
```

Finally we need to talk about the damping ratio. Do you already have a rough estimate of the damping value? Is it 0.1%, 1% or 10% damping. We can try to answer this by fitting the exponential decay of the impulse response to the data.

Assignment 1.5: Using the widget below can you match the damping ratio of the wind turbine? What value do you find, for the damping? Does that seem possible?

In [10]:

```

from ipywidgets import interact, fixed
from matplotlib.widgets import Slider, Button

# Define initial parameters
init_xi = 2

# Create the figure and the line that we will manipulate
fig, ax = plt.subplots()

t = np.arange(start=0, stop=100, step=0.1)

plt.plot(t_rotor_stop, fa_rotor_stop)
line, = plt.plot(t_rotor_stop, np.exp(-init_xi/100*w_n*t_rotor_stop)*r
ax.set_xlabel('Time (s)')
ax.set_ylabel('h(t)')
plt.grid(which='both', linestyle=':')

# adjust the main plot to make room for the sliders
plt.subplots_adjust(left=0.25, bottom=0.4)

axmass = plt.axes([0.25, 0.1, 0.65, 0.03])
xi_slider = Slider(
    ax=axmass,
    label='Damping ratio  $\xi$  (%)',
    valmin=0.01,
    valmax=3,
    valinit=init_xi,
)

# The function to be called anytime a slider's value changes
def update(fixed):
    h = np.exp(-xi_slider.val/100*w_n*t_rotor_stop)*max(fa_rotor_stop
    line.set_ydata(h)

    ax.relim()
    ax.autoscale_view()
    fig.canvas.draw_idle()

# register the update function with each slider
xi_slider.on_changed(update)

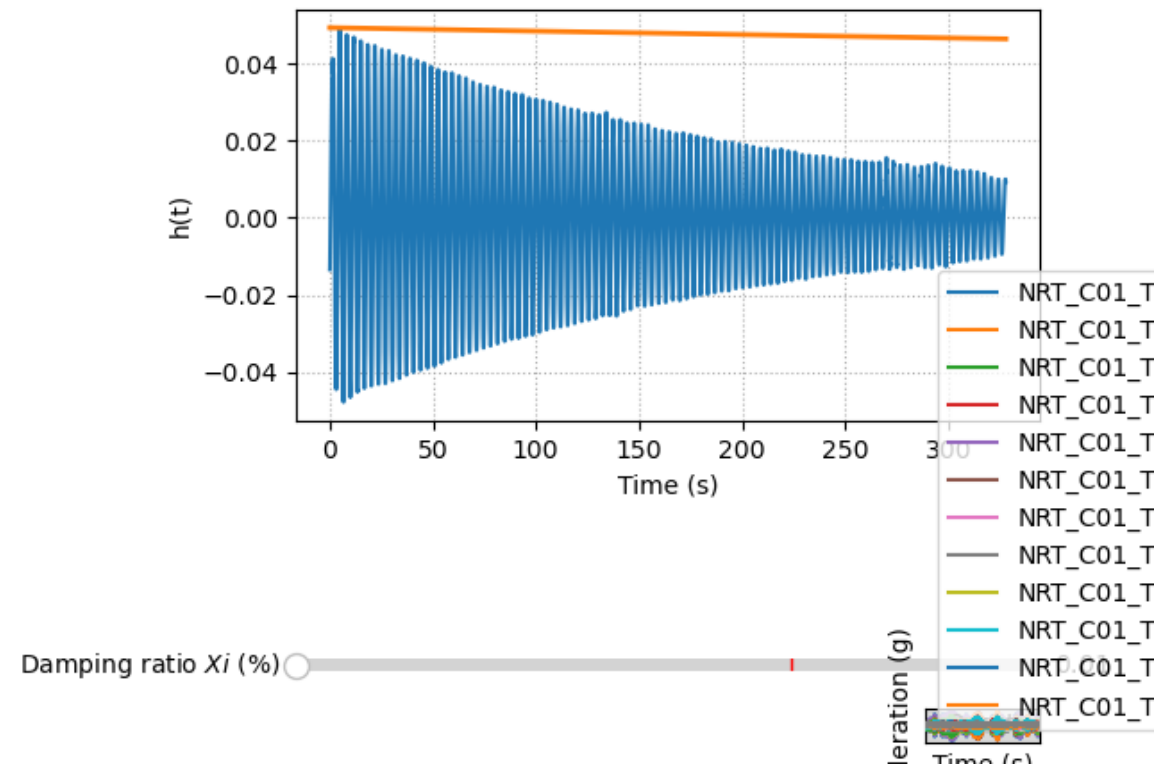
# Create a `matplotlib.widgets.Button` to reset the sliders to initial
resetax = plt.axes([0.8, 0.025, 0.1, 0.04])
button = Button(resetax, 'Reset', hovercolor='0.975')

def reset(event):
    xi_slider.reset()
button.on_clicked(reset)

w = interact(update(fixed))

```

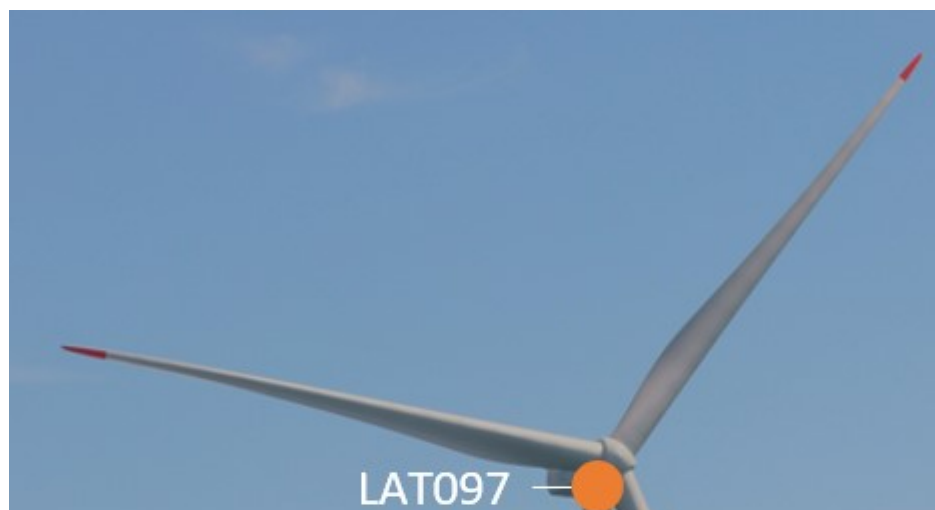
↳ Python core display javascript objects



Exercise 2 : the parked turbine

In the field of offshore wind energy the behaviour of a parked turbine (i.e. when it is not producing power) is almost equally important as the behaviour during operating conditions. At some sites the fatigue life is actually determined by the amount of time a turbine spends in parked conditions. A statement like "10% of downtime accounts for 90% of the fatigue life" are not uncommon.

In this exercise we will go to a **different wind farm**, and study the behaviour a turbine that is parked. This time we will have 6 measurement signals, recorded at three different heights of the turbine. In offshore wind turbines the height is often related to a reference called LAT or Lowest Astronomical Tide, the lowest occurring tidal level. So a sensor labeled LAT015 is installed 15 meters above this reference level, and a sensor labeled LAT097 is installed 97m above this reference level.





Assignment 2.1: Can you use pandas to import the measurement data that is stored in the file `offshore_wind_turbine_parked.csv` ? Store the import in the variable `df`

```
In [65]: ###BEGIN SOLUTION
df = pd.read_csv('offshore_wind_turbine_parked.csv', header=0)
df.head()
###END SOLUTION
```

```
Out[65]:
```

	t [s]	LAT015_FA [g]	LAT015_SS [g]	LAT069_FA [g]	LAT069_SS [g]	LAT097_FA [g]	LAT097_SS [g]
0	0.000000	-0.000116	0.000933	-0.003958	-0.000331	-0.005789	-0.000708
1	0.033333	-0.000130	0.000880	-0.003935	-0.000378	-0.005720	-0.000597
2	0.066667	-0.000189	0.000795	-0.003988	-0.000392	-0.005695	-0.000540
3	0.100000	-0.000274	0.000691	-0.004161	-0.000350	-0.005655	-0.000540
4	0.133333	-0.000357	0.000587	-0.004419	-0.000322	-0.005550	-0.000573

Let us check your import

```
In [66]: assert len(df.columns) == 7, f'Did you import the new data? Your impo
```

Assignment 2.2: Use the cell below to find key information about the measurement dataset. Define:

- The sample frequency in Hz: ``f_s``
- The duration in s: ``t_e``
- The measurement unit of the acceleration: ``unit_str``

```
In [80]: ###BEGIN SOLUTION
f_s = 30 #Hz
t_e = 600 #s
unit_str = 'g'
###END SOLUTION
```

Some small checks

```
In [81]: assert f_s*t_e == len(df), f'Something does not add up, the data conf
```

Assignment 2.3: use the `plt.plot` command to visualize the measurement data. Can you make sure to plot all 6 measurement signals?

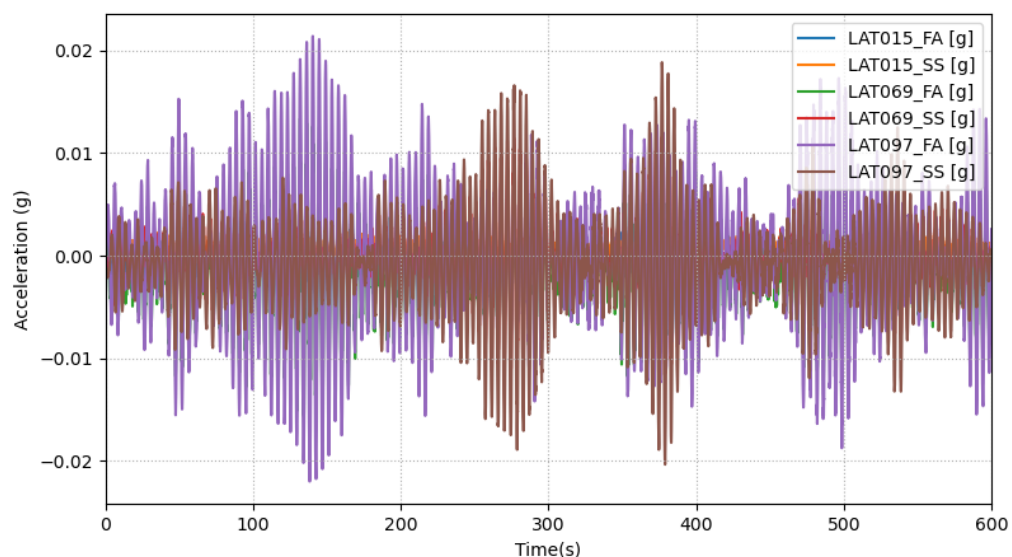
```
In [71]: plt.figure(figsize=(9,5))

###BEGIN SOLUTION
for c in list(df.columns)[1:]:
    plt.plot(df['t [s]'],df[c], label=c,zorder=1)

plt.xlim([0,600])
plt.legend()
plt.grid(which='both', linestyle=':')
plt.xlabel('Time(s)')
plt.ylabel(f'Acceleration ({unit_str})')
###END SOLUTION

plt.show()
```

<IPython.core.display.Javascript object>



This timeseries is a bit harder to interpret than that of the previous turbine. Perhaps transforming it into the frequency domain is more helpful in interpreting the results....

Assignment 2.4: Convert the measurement data to the frequency domain using `np.fft.fft` and `np.fft.fftfreq` and plot the resulting spectra. Use these spectra to estimate the first natural frequency of the structure. Zoom in on the range between 0 and

2 H7

```
In [86]: plt.figure(figsize=(9,5))

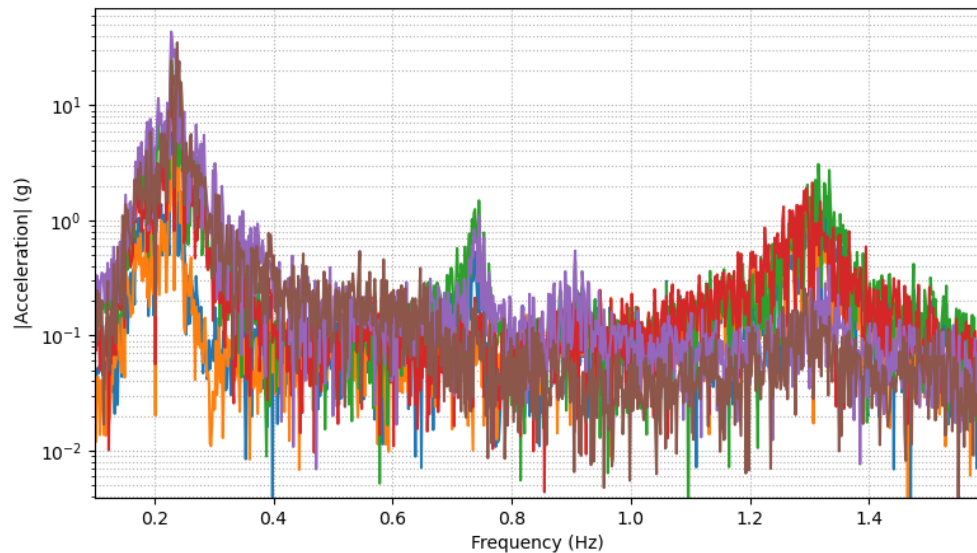
####BEGIN SOLUTION

f = np.fft.fftfreq(len(df['t [s]']), 1/f_s)
for c in list(df.columns)[1:]:
    X = np.fft.fft(df[c])

    fh = int(len(f)/2)
    plt.semilogy(f[:fh], np.abs(X[:fh]))
plt.xlim([0,2])
####END SOLUTION

plt.xlabel('Frequency (Hz)')
plt.ylabel('|Acceleration| (g)')
plt.grid(which='both', linestyle=':')
plt.show()
```

<IPython.core.display.Javascript object>

**Assignment 2.4:** Try to answer following questions below.

- How many modes can you spot between 0 and 2Hz?
- At which frequencies are they located?
- Do you observe differences between the different sensor levels? Can you spot all modes at every level?

In []:

In []:

